syslog-ng Store Box 7.7.0

RPC API Quickstart Guide

Copyright 2025 One Identity LLC.

ALL RIGHTS RESERVED.

This guide contains proprietary information protected by copyright. The software described in this guide is furnished under a software license or nondisclosure agreement. This software may be used or copied only in accordance with the terms of the applicable agreement. No part of this guide may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of One Identity LLC .

The information in this document is provided in connection with One Identity products. No license, express or implied, by estoppel or otherwise, to any intellectual property right is granted by this document or in connection with the sale of One Identity LLC products. EXCEPT AS SET FORTH IN THE TERMS AND CONDITIONS AS SPECIFIED IN THE LICENSE AGREEMENT FOR THIS PRODUCT, ONE IDENTITY ASSUMES NO LIABILITY WHATSOEVER AND DISCLAIMS ANY EXPRESS, IMPLIED OR STATUTORY WARRANTY RELATING TO ITS PRODUCTS INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL ONE IDENTITY BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE, SPECIAL OR INCIDENTAL DAMAGES (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF PROFITS, BUSINESS INTERRUPTION OR LOSS OF INFORMATION) ARISING OUT OF THE USE OR INABILITY TO USE THIS DOCUMENT, EVEN IF ONE IDENTITY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. One Identity makes no representations or warranties with respect to the accuracy or completeness of the contents of this document and reserves the right to make changes to specifications and product descriptions at any time without notice. One Identity does not make any commitment to update the information contained in this document.

If you have any questions regarding your potential use of this material, contact:

One Identity LLC.
Attn: LEGAL Dept
4 Polaris Way
Aliso Viejo, CA 92656

Refer to our website (http://www.OneIdentity.com) for regional and international office information.

Patents

One Identity is proud of our advanced technology. Patents and pending patents may apply to this product. For the most current information about applicable patents for this product, please visit our website at http://www.OneIdentity.com/legal/patents.aspx.

Trademarks

One Identity and the One Identity logo are trademarks and registered trademarks of One Identity LLC. in the U.S.A. and other countries. For a complete list of One Identity trademarks, please visit our website at www.OneIdentity.com/legal/trademark-information.aspx. All other trademarks are the property of their respective owners.

Legend

WARNING: A WARNING icon highlights a potential risk of bodily injury or property damage, for which industry-standard safety precautions are advised. This icon is often associated with electrical hazards related to hardware.

CAUTION: A CAUTION icon indicates potential damage to hardware or loss of data if instructions are not followed.

syslog-ng Store Box RPC API Quickstart Guide
Updated - 19 November 2025, 13:12

For the most recent documents and product information, see Online product documentation.

Contents

Introduction **4**

Authenticating **6**

Getting the list of logspaces **8**

Fetching the number of hits **9**

Searching **10**

Filtering for time **13**

Fetching statistics **14**

Logging out **16**

Importing and applying a configuration bundle **17**

About us **20**

Contacting us 20

Technical support resources 20

1

Introduction

This document will guide you through the first steps of using the RPC API. By following these steps you will get a good overview of the capabilities and the basic functionalities of the API.

Prerequisites

Previous experience:

- You are familiar with syslog-ng Store Box and its search interface
- You are familiar with related basic concepts, such as an API, REST, HTTP and JSON
- The examples are provided in the form of UNIX shell commands, we assume that you are comfortable using such an environment

Software requirements:

- Pre-installed syslog-ng Store Box that has the RPC API feature enabled.
- The "wget" utility is installed. It is widely available for the majority of platforms, and can be installed from http://www.gnu.org/software/wget/.
- The "jq" utility for command-line JSON formatting and conversion is installed. The source code and binaries for the most common platforms can be downloaded from http://stedolan.github.io/jq/.

Installation

Make sure you have your syslog-ng Store Box installed, configured and running. Throughout this guide we assume that it is accessible at the IP address `1.2.3.4` and that `admin/a` is a valid username/password combination.

We perform the following operations on the built-in local logspace, so make sure the defaults are not changed and it is configured to receive and index local logs.

The basics

The API to access the logs is a REST-based API that runs over HTTPS. All examples use the wget utility to access it, and we perform HTTP GET queries against SSB. Of course, mature libraries or built-in methods are available for practically all programming environments to interact with a REST-based API providing replies in JSON format, but throughout this guide we use the command line, performing the query with wget and processing the results with a tiny but useful utility called "jq".

A typical query looks like this:

$ wget -q --no-check-certificate --header [auth info, see later] -O - 'https://[IP address of SSB]/api/[API version]/[API command group]/][API command]?[API command arguments] | jq '[filters for the JSON output']

Documentation of the RPC API

The documentation of the SSB RPC API is available online from the following URL: *https://<ip-address-of-SSB>/api/4/documentation*. This documentation contains the detailed description of public calls, with examples.

2

Authenticating

Queries going through the API go through the same authentication-authorization-accounting steps as sessions using the UI. There are no special API permissions or users: the sames credentials are used for access over the API. To authenticate yourself, you have to use the login command. It will provide you with an authentication token which is valid until your session times out or you explicitly log off. You have to send back this authentication token in the header of all further requests. In the current beta version this token needs to be sent back as the PHP session ID cookie but this might change in the final version.

SSB prevents brute force attacks when logging in. If you repeatedly try logging in to SSB using incorrect login details within a short period of time (10 times within 60 seconds), the source IP gets blocked on UI destination port 443 for 5 minutes. Your browser displays an **Unable to connect** page.

1. Let's log in. Remember, we assume that `1.2.3.4` is the IP address of your SSB and `admin/a` is a valid username/credential pair. Make sure you alter your query according to your environment.

   $ SSB_IP=1.2.3.4

   $ wget -q --no-check-certificate -O - "https://$SSB_IP/api/5/login" --post-data='username=admin&password=a'

   ```
   {"result":"af574226cb4d3de28312b27f2bfc705e","error":
   {"code":null,"message":null},"warnings":[]}
   ```

2. OK, this is a bit hard to read, let's have jq pretty-print it:

   $ wget -q --no-check-certificate -O - "https://$SSB_IP/api/5/login" --post-data='username=admin&password=a' | jq '.'

   ```
   {
     "warnings": [],
     "error": {
   ```

```
    "message": null,
    "code": null
  },
  "result": "39d5b0229ff744cdd384d54618d7039c"
}
```

That's better, and now the structure of responses can be seen. It is always a single JSON object with three parts: warnings, error (these first two are optional and might be omitted when not in used in the final version) and the most important part, the result block. In this case it contains our identifier key. Note that it has changed since the first query: as we logged in again it has been regenerated and the old key is no longer accepted.

3. We need it for later examples, so let's save it into a variable. We use the filtering capabilities of jq to cut out the part we are interested in: the result block itself.

   $ SESSID=`wget -q --no-check-certificate -O - "https://$SSB_IP/api/5/login" --post-data='username=admin&password=a' | jq '.result' -r`

   $ echo $SESSID

   ```
   f36a9399c3cbcc3ca5dab037f8374dd8
   ```

   Now we can move on to perform real queries.

Getting the list of logspaces

1. Maybe you know which logspace you want to query, maybe you don't. You can query the list of accessible logspaces with the list_logspaces command:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/list_logspaces" | jq '.result'

   ```
   [
       "local",
       "center"
   ```

   As you can see, there are only these two logspaces available on this SSB. We investigate the "local" logspace from now on, as by default, that contains some well-known messages even after a fresh installation, which we can use in our examples.

Fetching the number of hits

In some cases you might not need the actual results, only their number. To draw charts (as we do in the timeline section of our search UI) or to calculate statistics, usually the number of results is enough. You could always perform a search, fetch all results, and then count their number, but that's not really effective. Instead, you can ask the API to only return the number of hits by replacing **filter** with **number_of_messages** in your request and remove the offset/limit settings.

1. To follow up on the previous example, let's see the number of messages containing "starting up" in overall and the number of those sent out by syslog-ng:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_messages/local?from=0&to=9999999999&search_expression=starting up" | jq '.result'

   ```
   84
   ```

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_messages/local?from=0&to=9999999999&search_expression=starting up program:syslog-ng" | jq '.result'

   ```
   1
   ```

5

Searching

OK, let's do some real searching now. The command for that is **filter**, and the arguments that have to be specified in sequence, separated by slashes are:

- name of the logspace
- from (as a UNIX timestamp)
- to (timestamp, too)
- your search expression (optional, defaults to none)
- offset
- limit (these work just as they do in SQL)

1. At first let's find when was syslog-ng started or restarted in the box:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/filter/local?from=0&to=9999999999&search_expression=starting up&offset=0&limit=10" | jq '.result'

```
[
  {
    "tag": [],
    "dynamic": {
      ".SDATA.timeQuality.isSynced": "0"
    },
    "msgid": "",
    "stamp": 1384943027,
    "recvd": 1384943027,
    "pri": 5,
    "facility": 5,
    "host": "rpcapitest",
    "message": "logindexd starting up; version='4.2.4ssb3.2.23'",
    "program": "index-center",
    "pid": "4408"
  },
  {
```

```
    "tag": [],
    "dynamic": {
      ".SDATA.timeQuality.isSynced": "0"
    },
    "msgid": "",
    "stamp": 1384943027,
    "recvd": 1384943027,
    "pri": 5,
    "facility": 5,
    "host": "rpcapitest",
    "message": "logindexd starting up; version='4.2.4ssb3.2.23'",
    "program": "index-local",
    "pid": "4407"
  },
[..... and a screenful of other results .....]
```

As you can see I set the timestamps to 0 and some huge value to make sure time filtering does not apply. We will try that in a sec, too.

2. But first, this doesn't seem right, it's not only messages from syslog-ng, the indexer processes are talking as well. Let's only print the program messages from the result set with jq to see clearer:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/filter/local?from=0&to=9999999999&search_expression=starting up&offset=0&limit=10" | jq '.result[].program'

```
"index-center"
"index-local"
"syslog-ng"
"index-local"
"index-center"
"index-local"
"index-local"
"index-local"
"index-local"
"index-local"
```

3. Let's alter our query to make sure we filter for the messages coming from syslog-ng only:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/filter/local?from=0&to=9999999999&search_expression=starting up program:syslog-ng&offset=0&limit=10" | jq '.result'

```
[
    {
        "tag": [],
        "dynamic": [],
        "msgid": "",
        "stamp": 1384943029,
        "recvd": 1384943029,
        "pri": 5,
        "facility": 5,
        "host": "ssb1",
        "message": "syslog-ng starting up; version='4.2.4ssb3.2.23', cfg-
fingerprint='54d093e5b748276c497a141390614e079fff6cc0', cfg-nonce-ndx='0',
cfg-signature='d46966469bf5934c1c7280375f349dcc2d7906dc'",
        "program": "syslog-ng",
        "pid": "4433"
    }
```


It's much better now, as you can see it is a pretty fresh install and syslog-ng has only been started once. The only thing we did is that we added `program:syslog-ng` to our query. You can use everything there that you could use on the UI. Note that wget actually helps a lot here: it is automatically URL-encoding our query string, making sure that special characters such as the space and the colon are passed properly to the API. In other environments, you might have to do that yourself.

> **TIP:**
>
> To decrease the load on SSB when searching and receive your search results faster, note the following points.
>
> - Use as small a time range as possible
> - Prefer AND instead of OR
> - Avoid unneeded wildcard characters, such as `*` and `?`
> - Use wildcard characters at the end of the tokens if possible.

Filtering for time

In the previous examples, we filtered our results by expressions and made sure that no time limitation applied. Let's turn that the other way around and only filter for time.

1. Let's see the number of logs in the local logspace in a given five minute interval:

   $ date +%s -d"Nov 20, 2013 12:55:00"

   ```
   1384948500
   ```

   $ date +%s -d"Nov 20, 2013 13:00:00"

   ```
   1384948800
   ```

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_messages/local?from=1384948500&to=1384948800" | jq '.result'

   ```
   38
   ```

2. You can of course combine that with regular filtering. Here we get the number of logs produced by indexer processes in that timerange.

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_messages/local?from=1384948500&to=1384948800&search_expression=program:index*" | jq '.result'

   ```
   7
   ```

   Note that I used a wildcard character in the query: that's possible, too. Pay attention of the URL-encoding and escaping in your environment if you do.

Fetching statistics

In our first example, we investigated which programs issued messages containing the words "starting" and "up". We could have created some statistics simply by using standard command line tools:

$ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/filter/local?from=0&to=9999999999&search_expression=starting up&offset=0&limit=1000" | jq '.result[].program' -r | sort | uniq -c

```
      2 index-center
     96 index-local
      1 syslog-ng
```

However, that's terribly ineffective: we had to pass all messages over the network and use the computing capabilities of our client machine to do the aggregation. This is feasible when we are talking about such numbers but not when there are billions of entries. In that case, it is a much better idea to use the built-in statistics engine of syslog-ng Store Box.

1. To have SSB generate you the same statistics, you can use the **generate_statistics** command. These are its arguments:
   - name of the logspace
   - column to generate statistics from
   - from (as a UNIX timestamp)
   - to
   - your search expression (optional, defaults to none)
   - offset (optional)
   - limit (optional)

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/generate_statistics/local/program?from=0&to=9999999999&search_expression=starting up" | jq '.result'

```
[
    [
       "syslog-ng",
       1
    ],
    [
       "index-center",
       2
    ],
    [
       "index-local",
       97
    ]
```

2. As the number of entries here can also be huge, this command has the same offset/limit possibilities that the **filter** command has, and you can also fetch the number of entries (distinct values if you wish) with the **number_of_statistics_entries** command:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_statistics_entries/local/program?from=0&to=9999999999&search_expression=starting up" | jq '.result'

```
3
```

Logging out

1. And of course your should finish your session by logging out:

   $ wget -q --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKE=$SESSID" "https://$SSB_IP/api/5/logout" | jq '.'

   ```
   {
   "warnings": [],
   "error": {
   "message": null,
   "code": null
   },
   "result": true
   }
   ```

2. Now the old session ID won't work any more and you'll get a 403 error when you try to perform a query:

   $ wget --no-check-certificate -O - --header "Cookie: AUTHENTICATION_TOKEN=$SESSID" "https://$SSB_IP/api/5/search/logspace/number_of_statistics_entries/local/program?from=0&to=9999999999&search_expression=starting up"

   ```
   --2014-01-07 14:25:21--  https://1.2.3.4/api/5/search/logspace/number_of_
   statistics_entries/local/program?from=0&to=9999999999&search_
   expression=starting%20up
   Connecting to 1.2.3.4:443... connected.
   WARNING: cannot verify 1.2.3.4's certificate, issued by
   `/C=AT/L=asdfasdf/O=rtyrty/OU=cbdfg/ST=ertertert/CN=rpcapitest.gamma.test.
   balabit root CA':
   Self-signed certificate encountered.
   HTTP request sent, awaiting response... 403 Forbidden
   2014-01-07 14:25:21 ERROR 403: Forbidden.
   ```

9

Importing and applying a configuration bundle

This section describes a REST API version of importing a previously exported configuration bundle in SSB. Throughout this section, the examples of transferring data between SSB and the client use the command `curl` instead of `wget`.

> **IMPORTANT:** In case of using unofficial root CA certificates in SSB for HTTPS communication, the default behavior of command `curl` is to return an `Unknown CA` error message. To resolve this error, use option `--insecure` with `curl`.

The examples in the procedure use the following values:

- SSB IP address: `1.2.3.4`
- Authentication token (resulted from the login): `0123456789abcdef`
- Filename of the configuration bundle: `config_bundle.config`
- MD5 hash value of the configuration bundle: `a1b2c3d4f5g6h7i8j9`

To import a previously exported configuration bundle in SSB via REST API

1. Upload and validate the previously exported configuration bundle.

   `/config/upload_and_validate`

   Method: **`POST`**

   Post parameters: (Optional) **`config_password`**

   Uploads, then validates and retrieves the summary of the current uploaded candidate configuration bundle.

   Optional parameter: the password of the encrypted configuration bundle.

   > **NOTE:** If the bundle is not password encrypted, you can skip the `config_password`.

   Command-specific response:

```
{
    'original_filename': <original bundle filename>,
    'original_file_md5': <md5 hash of original bundle file>,
    'Hostname': <hostname in candidate bundle>,
    'Domainname': <domainname in candidate bundle>,
    'External address': <external IP address in candidate bundle>,
    'Management address': <management IP address in candidate bundle>,
    'Netmask': <netmask in candidate bundle>,
    'Default gateway': <default gateway in candidate bundle>,
    'DNS server': <DNS server in candidate bundle>,
    'Timezone': <timezone in candidate bundle>,
    'NTP server': <NTP server in candidate bundle>,
    'SSL certificate': <SSL certificate in candidate bundle>,
    'SSH RSA key': <SSH RSA key in candidate bundle>,
    'Licensed version': <license version in candidate bundle>,
    'Licensed customer': <licensed customer in candidate bundle>,
    'Licensed hosts': <licensed hosts in candidate bundle>,
    'License serial': <license serial in candidate bundle>,
    'Config md5 hash': <md5 hash of uploaded configuration bundle>
}
```

Example: Uploading and validating a configuration bundle

```
$ curl --request POST --header 'Cookie: AUTHENTICATION_
TOKEN=0123456789abcdef' --form 'config=@config_bundle.config' --form
'config_password=secret_bundle_password'
'https://1.2.3.4/api/5/config/upload_and_validate'
```

2. (Optional) Validate the current configuration bundle.

   /config/validate

   Method: **GET**

   Validates and retrieves the summary of the current uploaded candidate configuration bundle, if there is one.

   Command-specific response:

```
{
    'original_filename': <original bundle filename>,
    'original_file_md5': <md5 hash of original bundle file>,
    'Hostname': <hostname in candidate bundle>,
    'Domainname': <domainname in candidate bundle>,
    'External address': <external IP address in candidate bundle>,
    'Management address': <management IP address in candidate bundle>,
    'Netmask': <netmask in candidate bundle>,
    'Default gateway': <default gateway in candidate bundle>,
    'DNS server': <DNS server in candidate bundle>,
```

```
    'Timezone': <timezone in candidate bundle>,
    'NTP server': <NTP server in candidate bundle>,
    'SSL certificate': <SSL certificate in candidate bundle>,
    'SSH RSA key': <SSH RSA key in candidate bundle>,
    'Licensed version': <license version in candidate bundle>,
    'Licensed customer': <licensed customer in candidate bundle>,
    'Licensed hosts': <licensed hosts in candidate bundle>,
    'License serial': <license serial in candidate bundle>,
    'Config md5 hash': <md5 hash of uploaded configuration bundle>
}
```

Example: Validating the current configuration bundle

```
$ curl --request GET --header 'Cookie: AUTHENTICATION_
TOKEN=0123456789abcdef' 'https://1.2.3.4/api/5/config/validate'
```

3. To make the changes take effect, apply the configuration.

   /config/apply

Example: Applying the uploaded configuration bundle

```
$ curl --request POST --header "Cookie: AUTHENTICATION_
TOKEN=0123456789abcdef" 'https://1.2.3.4/api/5/config/apply_config' -
-form 'config_md5=a1b2c3d4f5g6h7i8j9'
```

Sufficient condition: The user uploading the configuration bundle must be in a group with **Import configuration** access control rights. For more information about configuring group access control, see *Managing user rights and usergroups in the syslog-ng Store Box Administrator Guide*.

About us

One Identity solutions eliminate the complexities and time-consuming processes often required to govern identities, manage privileged accounts and control access. Our solutions enhance business agility while addressing your IAM challenges with on-premises, cloud and hybrid environments.

Contacting us

For sales and other inquiries, such as licensing, support, and renewals, visit https://www.oneidentity.com/company/contact-us.aspx.

Technical support resources

Technical support is available to One Identity customers with a valid maintenance contract and customers who have trial versions. You can access the Support Portal at https://support.oneidentity.com/.

The Support Portal provides self-help tools you can use to solve problems quickly and independently, 24 hours a day, 365 days a year. The Support Portal enables you to:

- Submit and manage a Service Request
- View Knowledge Base articles
- Sign up for product notifications
- Download software and technical documentation
- View how-to videos at www.YouTube.com/OneIdentity
- Engage in community discussions
- Chat with support engineers online
- View services to assist you with your product